

Time Dependent Quantum Mechanics

In Lecture 12, we developed solutions for the stationary Schrödinger equation. We solved the harmonic oscillator and the particle in a box model. All of this provided a static picture rather than the dynamics of particles in potentials. Today we will explore the time-dependent Schrödinger equation which yields the dynamics of quantum systems. We will develop solutions based on Fourier transforms, building upon the stationary solutions and extending them to understand how quantum states evolve over time.

```
#| edit: false
#| echo: false
#| execute: true

#| slideshow: {slide_type: skip}
#| tags: []

import numpy as np
import matplotlib.pyplot as plt
from scipy.constants import *
from scipy.sparse import diags
from scipy.sparse.linalg import eigsh
from scipy.fftpack import fft,ifft
from time import sleep,time

# Physical constants used throughout the lecture
hbar = 1.0545718e-34 # Reduced Planck constant in J·s
electron_mass = 9.10938356e-31 # Electron mass in kg

# Set default plotting parameters
plt.rcParams.update({
    'font.size': 12,
    'lines.linewidth': 1,
    'lines.markersize': 5,
```

```

    'axes.labelsize': 11,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'xtick.top': True,
    'xtick.direction': 'in',
    'ytick.right': True,
    'ytick.direction': 'in',
})

def get_size(w, h):
    return (w/2.54, h/2.54)

```

Time dependent Schrödinger equation

The time dependent Schrödinger equation is given by

$$-i\hbar \frac{\partial \Psi}{\partial t} = -\frac{\hbar^2}{2m} \nabla^2 \Psi + V(\vec{r}) \Psi \quad (1)$$

in 3 dimensions. We have already briefly mentioned this equation in 1 dimension

$$-i\hbar \frac{\partial \Psi(x, t)}{\partial t} = \left(-\frac{\hbar^2}{2m} \frac{\partial^2}{\partial x^2} + V(x, t) \right) \Psi(x, t) \quad (2)$$

during the last lecture. We are using the 1-dimensional equation today to study the dynamics of a particle in a box and the tunneling of a particle. To do so, we first need to understand wavepackets.

Wavepackets

Physical Interpretation of Wavepackets

In quantum mechanics, particles are described by wavefunctions. However, a pure plane wave extends infinitely in space, which doesn't represent a localized particle. To describe a localized quantum particle, we need to construct a **wavepacket** - a superposition of plane waves with different wavelengths that interfere constructively in one region of space and destructively elsewhere, creating a localized probability distribution.

A wavepacket can be constructed from a superposition of plane waves with different wavenumbers k or different frequencies ω . The mathematical representation is:

$$\Psi(x, t) = A \int_{-\infty}^{+\infty} \phi(k) e^{ikx} e^{-i\omega t} dk \quad (3)$$

The function $\phi(k)$ represents the amplitude distribution in k -space (momentum space), giving the contribution of each wavenumber k to the wavepacket. For example, if only one wave at k_0 contributes to the wavepacket, then $\phi(k) = \delta(k - k_0)$ (a Dirac delta function), and the integral results in a single plane wave. For a real particle, $\phi(k)$ has a non-zero width, representing uncertainty in the particle's momentum.

For matter waves, the dispersion relation connecting ω and k is:

$$\omega = \frac{\hbar k^2}{2m} \quad (4)$$

Demonstration of superposition of plane waves

The code below demonstrates how waves with different wave vectors can be combined to result in a wave packet, i.e., an object that is localized in space. This is essentially a Fourier decomposition of a spatially extended object.

```
#| autorun: false
wavelength = 1e-10 # wavelength of the matter wave in meters
k0 = 2*np.pi/wavelength # central wavevector of the matter wave
dk = k0/10 # width of the wavevector distribution
electron_mass = 9.10938356e-31
hbar = 1.0545718e-34
```

```
#| autorun: false
plt.figure(figsize=get_size(12,6))

x = np.linspace(-2e-9, 2e-9, 1000)
k = k0
t = 0
frequency = hbar*k**2/(2*electron_mass)
h = np.exp(1j*(k*x-frequency*t))
a = np.copy(h)

plt.plot(x, np.real(h), label='$k_0$')

k = k0+dk
```

```

frequency = hbar*k**2/(2*electron_mass)
h = np.exp(1j*(k*x-frequency*t))
a += h
plt.plot(x, np.real(h)+2, label='$k_0+dk$')

k = k0-dk
frequency = hbar*k**2/(2*electron_mass)
h = np.exp(1j*(k*x-frequency*t))
a += h
plt.plot(x, np.real(h)+4, label='$k_0-dk$')

plt.plot(x, np.real(a)+8, label='$superposition$')
plt.axvline(x=0)
#plt.legend(loc='upper right')
plt.xlim(-2e-9, 2e-9)
plt.xlabel('position in [m]')
plt.ylabel('amplitude')

plt.tight_layout()
plt.show()

```

Rectangular Wavepacket

If the amplitude distribution $\phi(k)$ is constant over a range from $k_0 - \Delta k/2$ to $k_0 + \Delta k/2$ (i.e., a rectangular distribution in k-space), we get our first type of wavepacket.

To calculate the shape of the wavepacket, we can approximate the dispersion relation by:

$$\omega \approx \omega_0 + \frac{\partial \omega}{\partial k}(k - k_0) \quad (5)$$

which replaces the quadratic dispersion relation with an approximate linear one valid near k_0 .

After integrating over all contributing wavenumbers, we obtain the wavepacket:

$$\Psi(x, t) = 2A \frac{\sin \left[\left(x - \frac{\partial \omega}{\partial k} t \right) \frac{\Delta k}{2} \right]}{x - \frac{\partial \omega}{\partial k} t} e^{i(k_0 x - \omega_0 t)} \quad (6)$$

This wavepacket has two important velocities:

1. The **phase velocity** $v_{ph} = \omega_0/k_0$, which describes how the phase of the carrier wave propagates
2. The **group velocity** $v_g = \partial\omega/\partial k$, which describes how the envelope of the wavepacket moves

For matter waves, $v_g = \hbar k_0/m$, which is exactly the classical velocity of a particle with momentum $p = \hbar k_0$. This illustrates an important connection between quantum and classical mechanics - the group velocity of a wavepacket corresponds to the classical velocity of the particle.

Wavepacket with rectangular amplitude

The wavepacket above had just constant amplitudes over a range of wavenumbers. While we have an analytical function for this type of wave, we just program a function to show that the superposition works as well.

```
#!/ autorun: false
# matter wave packet with a frequency that scales with the square of the wave vector
def matter_wave_packet(x, t):
    h = 0
    N = 100
    for k in np.linspace(k0-dk/2, k0+dk/2, N):
        frequency = hbar*k**2/(2*electron_mass)
        h += np.exp(1j*(k*x-frequency*t))/N
    return h
```

The plot below shows the probability density $|\Psi|^2$. Notice the side lobes besides the main peak - this pattern is reminiscent of the diffraction pattern of a single slit in optics. This similarity is not coincidental; both phenomena arise from the same mathematical structure: the Fourier transform of a rectangular function yields a sinc function.

```
#!/ autorun: false
fig = plt.figure(1, figsize=get_size(12,6))

x = np.linspace(-10e-9, 10e-9, 2000)
plt.plot(x, np.abs(matter_wave_packet(x, 0))**2)
plt.xlabel(r'position [m]')
plt.ylabel(r'probability density  $|\Psi|^2$ ')
plt.tight_layout()
plt.show()
```

Gaussian Wave Packet

A more common and mathematically convenient wavepacket has a Gaussian distribution of amplitudes $\phi(k)$. When the amplitudes of the interfering plane waves follow a Gaussian distribution in k-space, the resulting spatial distribution is also Gaussian. This type of wavepacket has the special property that it maintains its Gaussian shape as it evolves over time, though it may spread out.

```
#| autorun: false
def gauss_wave_packet(x, t):
    h = 0
    sigma = np.sqrt(2)*dk # width in k-space
    for k in np.linspace(k0-5*dk, k0+5*dk, 1000):
        frequency = hbar*k**2/(2*electron_mass)
        h += np.exp(-(k-k0)/sigma)**2*np.exp(1j*(k*x-frequency*t))
    return h
```

```
#| autorun: false
fig = plt.figure(1, figsize=get_size(12,6))

x = np.linspace(-5e-9, 5e-9, 10000)
plt.plot(x, np.real(gauss_wave_packet(x, 0)), label='Real part')
plt.plot(x, np.abs(gauss_wave_packet(x, 0)), label='Amplitude')
plt.xlabel(r'position [m]')
plt.ylabel(r'amplitude')
plt.legend()

plt.tight_layout()
plt.show()
```

The Uncertainty Principle

The width of a wavepacket in position space (Δx) and its width in k-space (Δk) are inversely related, a manifestation of Heisenberg's uncertainty principle:

$$\Delta x \cdot \Delta k \geq \frac{1}{2} \quad (7)$$

Since momentum $p = \hbar k$, this becomes:

$$\Delta x \cdot \Delta p \geq \frac{\hbar}{2} \quad (8)$$

This means that a wavepacket that is very localized in position space (small Δx) must have a broad distribution in k-space (large Δk), and vice versa. This trade-off is intrinsic to quantum mechanics and has profound implications for the behavior of quantum particles.

Time evolution of a Gaussian Wavepacket

As a wavepacket evolves in time, two main effects occur: 1. The center of the wavepacket moves at the group velocity v_g 2. The wavepacket spreads out due to dispersion (different frequency components moving at different velocities)

This spreading is a purely quantum effect with no classical analog, and it's why quantum particles can't remain perfectly localized indefinitely.

Here we show an animation of a wavepacket's evolution over time:

The next lines do the magic here. You see that there is essentially a normal `matplotlib` plotting in the first lines. The special lines are

```
plt.draw()
background = fig.canvas.copy_from_bbox(ax.bbox)
points=ax.plot(x*1e9,np.abs(gauss_wave_packet(x,t))**2)[0]
plt.close()
```

which draws the diagram without the data into the buffer. We store that buffer as the background with `fig.canvas.copy_from_bbox(ax.bbox)`, because it is every time the same. The third line obtains the data point list as a matplotlib object. This list can be later updated with new data for the animation. Finally, we close the figure with `plt.close()` to prevent extra drawing below the cell.

```
x=np.linspace(0,20e-9,1000)

fig, ax = plt.subplots(1,1,figsize=(5,2),dpi=150)
plt.xlim(0,20)
plt.ylim(0,100000)

plt.xlabel('x [nm]')
plt.ylabel(r'$|\Psi(x,t)|^2$')
plt.tight_layout()

plt.draw()
background = fig.canvas.copy_from_bbox(ax.bbox)
```

```
points=ax.plot(x*1e9,np.abs(gauss_wave_packet(x,t))**2)[0]
plt.close()
```

The animation is then done by a for loop through time calculating the new wavefunction data and supplying it to the points object as new data. In detail it looks this way.

```
for t in np.linspace(0,5e-15,100):
    fig.canvas.restore_region(background) ## restore the background we have saved before
    ax.draw_artist(points) ## draw the plot with the data points
    points.set_data(x*1e9,np.abs(gauss_wave_packet(x,t))**2) ## modify the data in the points

    fig.canvas.blit(ax.bbox) ## copy everything into the figure canvas

    X = np.array(fig.canvas.renderer.buffer_rgba()) ## obtain the image representing the plot

    ## this is now our previous ipycanvas image drawing
    with hold_canvas(canvas):
        canvas.clear()
        canvas.put_image_data(X)
        sleep(0.02)
```